PRISM Search Control Results Report

PLN and Openpsi experiments across initial implementation of prism

Executive conclusion

PRISM produced measurable search-control gains in the evaluated PLN settings. Goal-aware candidate selection alone changed depth-5 chain success from 0% to 100% at 10 and 25 distractors. With indexed premise filtering added, the recorded depth-8 and depth-10 noisy chain cases at 25 and 50 distractors also reached 100%. On a normalized 100-sentence external similarity graph, PRISM solved 5 of 5 held-out queries versus 2 of 5 for bounded confidence-driven PLN, while both systems used real PLN inference. Later proof replay verified 22 of 22 recorded derived actions in eight tested cases. [W2, W3, KG, W10]

The contribution is not one monolithic speedup. Candidate ranking, premise filtering, explicit agenda management, strategic subgoaling, and bidirectional search answer different control questions. Their comparisons use different runners and budgets, so this report keeps results grouped by experiment. The learned Tier 1 v2 checkpoint met its original parity and latency gates but did not outperform v1 rather returned similar results.

OpenPsi provides an independent planning example. On 21 goals from one repository conversation-rule graph, PRISM reached 20, a matched native v2 selector reached 3, and a separate default-v1 diagnostic reached 1. This is a plan-reachability result, not a demonstrated improvement in real agent task outcomes; the one effectful ping-pong comparison was a tie.

1 How to read the comparisons

- The early PeTTa PLN loop, the initial Python agenda, and the later real-PLN.Apply agenda are distinct implementations and experiments. Never combine their timing or expansion counts into a single speedup.
- In the early 80-step PLN loop, reported average steps were 80 even when the goal was found; that field is the full derive budget, not time to first proof. A faster failed run is not faster time to proof.
- Success percentages in the early chain sweep come from two repetitions per cell. Thus 50% means one success in two trials, not a population estimate.
- A valid derived result comes from PLN rules. Filtering and ranking can still miss a reachable proof under a budget; proof soundness and search completeness are separate questions.

Notation: D = chain depth; N = distractor facts; S = success rate; W = reported waste ratio; P = average distractor selections; T = average wall-clock seconds. Source keys appear in the final source index.

2 PLN native derivation loop

2.1 Hook, cost, and compatibility

The first integration threaded an optional goal through PLN.Query and PLN.Derive, then used PriorityRankGoal to choose inference tasks while lib_pln.metta retained rule application and truth-value computation. A goal-relevant task with confidence 0.80 outranked an irrelevant confidence-0.95 task. Reported py-call latency was 0.015 ms cached and 0.021 ms uncached. Exception fallback returned to raw confidence; seven rule tests and six standard examples passed. These are local integration measurements, not general throughput claims. [W1]

2.2 Candidate selection and indexed premise filtering

The Tier 1 v1 candidate selector is an independently measured improvement: in the same 80-step chain sweep, ranking alone rescued D=5 at N=10 and N=25 before Stage 0 was added. At D=5, N=25, it cut reported waste from 83.75% to 45.00%, distractor selections from 62.5 to 2.0, and mean elapsed time from 0.9782 s to 0.2323 s. Stage

0 then narrowed eligible premise pairs and complemented that ranking; it was not the sole source of the observed benefit. [W2, W3]

Full chain sweep. Each entry is S / W / P / T. Same nominal budget: 80 derivation steps, task queue 30, belief queue 100; two repetitions per cell. [W2, W3]

D	N	Unguided PLN	Tier 1 v1	Tier 1 v1 + Stage 0
5	0	100% / 35.00% / 0 / 0.7455	100% / 47.50% / 0 / 0.1429	100% / 47.50% / 0 / 0.2045
5	10	0% / 75.62% / 49.5 / 0.6505	100% / 47.50% / 0 / 0.1597	100% / 47.50% / 0 / 0.2355
5	25	0% / 83.75% / 62.5 / 0.9782	100% / 45.00% / 2 / 0.2323	100% / 45.00% / 0 / 0.3044
5	50	0% / 84.38% / 65 / 0.6668	50% / 47.50% / 4.5 / 0.2407	100% / 47.50% / 0 / 0.2949
8	0	0% / 31.25% / 0 / 1.0974	0% / 35.00% / 0 / 0.3886	100% / 43.75% / 0 / 0.7221
8	10	0% / 65.00% / 37.5 / 0.7039	0% / 30.63% / 0.5 / 0.4347	100% / 38.75% / 0 / 0.6987
8	25	0% / 73.12% / 57 / 0.9873	0% / 52.50% / 24 / 0.5285	100% / 45.00% / 0 / 0.3250
8	50	0% / 73.12% / 56 / 0.6576	0% / 55.62% / 26.5 / 0.4527	100% / 47.50% / 0 / 0.3092
10	0	0% / 32.50% / 0 / 1.1614	0% / 35.00% / 0 / 0.2126	0% / 42.50% / 0 / 0.3506
10	10	0% / 58.75% / 34 / 0.7283	0% / 27.50% / 0.5 / 0.2935	0% / 37.50% / 0 / 0.5615
10	25	0% / 65.62% / 50.5 / 0.9321	0% / 51.25% / 23.5 / 0.5085	100% / 45.00% / 0 / 0.3411
10	50	0% / 65.62% / 50 / 0.6444	0% / 49.38% / 24.5 / 0.4561	100% / 47.50% / 0 / 0.3234

The pattern is not monotonic: D=10 at N=0 or 10 still failed even with both controls, while some noisier cells succeeded. On clean D=5, Tier 1 increased the reported waste ratio from 35.00% to 47.50%. Stage 0 often increased elapsed time over Tier 1 alone at D=5, although it removed the last distractor picks. These counterexamples argue for more seeds and queue-order analysis, not for dismissing candidate selection. [W2, W3]

Stage 0 used concept-overlap retrieval and a fallback to the unfiltered beliefs when no match was found. The early shared-proof check reported identical final STV [0.60645, 0.16888] and evidence IDs 1-5 for guided and unguided variants. In this configuration, queue eviction deliberately remained confidence-based to avoid repeated FFI scoring during LimitSize. [W2, W3]

2.3 Multipath and confluent proofs

The next PLN-loop tests broadened topology. Guided search chose the shortcut route in 12 of 12 recorded diamond runs. On two-branch trees, it recovered several proofs that the unguided loop missed, while one harder tree condition defeated both. The tables retain regressions as well as wins. [W4]

Diamond routes: all listed cells report 100% success for both modes. U/G = unguided/guided; P = mean distractor picks. [W4]

Short/long	N	Path U/G	P U/G	Time U/G (s)
2/5	0	mixed / shortcut	0 / 0	1.19 / 0.42
2/5	10	shortcut / shortcut	36 / 0	0.89 / 0.42
2/5	25	shortcut / shortcut	59 / 37	1.04 / 1.11
3/6	0	shortcut / shortcut	0 / 0	1.48 / 1.46
3/6	10	shortcut / shortcut	36.5 / 14.5	0.88 / 1.11
3/6	25	shortcut / shortcut	54 / 34.5	1.06 / 1.12

The 2/5 clean diamond improved from 1.19 to 0.42 s, but guided execution was slower in three noisy rows. Some unguided runs also took the shortcut; the evidence is guided consistency, not exclusive discovery of that path. [W4]

Two-branch tree: S and P are success and distractor selections; U/G notation as above. [W4]

Tree	N	Success U/G	P U/G	Time U/G (s)
L(2,2)	0	100% / 100%	0 / 0	0.49 / 0.20
L(2,2)	10	100% / 100%	52 / 0	0.68 / 0.22
L(2,2)	25	0% / 100%	64.5 / 0	1.05 / 0.29

Tree	N	Success U/G	P U/G	Time U/G (s)
L(3,3)	0	0% / 100%	0 / 0	1.01 / 0.24
L(3,3)	10	0% / 100%	44 / 0	0.73 / 0.28
L(3,3)	25	0% / 0%	61 / 27	1.04 / 0.75

The tree gains concern proof reachability under the 80-step budget. Dividing the runtimes of a successful run by an unsuccessful baseline would not measure proof-speedup. Week 4 also logged structured proof traces for later learning, using evidence-stamp attribution to label selected steps. [W4]

3 Explicit search control

3.1 Initial unfiltered agenda

The explicit search manager added a heap-based open set, visited-state hashing, a cumulative uncertainty/path cost, a Tier 1 heuristic term, beam pruning, and proof-path reconstruction. Its first benchmark compared an unguided uniform-cost/FIFO-style agenda with a heuristic-guided A*-style agenda, not against the original PLN.Derive loop. It used a temporary local Python deduce_pair approximation; its search-efficiency numbers are historical and do not establish PLN semantic equivalence. Later work replaced that inference boundary with real PLN.Apply. [W5, C]

Initial agenda: seed 42, step budget 80, beam width 5. [W5]

Scenario	Success U/G	Expansions U/G	Nodes U/G	Time U/G (s)
Chain D4 N10	yes / yes	28 / 19	84 / 60	0.0148 / 0.0113
Diamond 3/7 N30	yes / yes	17 / 7	51 / 21	0.0270 / 0.0100
Chain D6 N25	no / no	80 / 80	not reported	not reported
Tree L(3,3) N20	no / no	80 / 80	not reported	not reported

Thus the first agenda reduced expansions by 32.1% on the solved D4 chain and 58.8% on the solved diamond. It did not rescue the two hard cases in this recorded configuration. “Learned A*” in the source heading refers to a heuristic-guided agenda at this stage; the learned neural scorer was introduced only in Week 11. [W5, W11]

3.2 Stage 0 inside the agenda and backward primitives

A separate later run wired concept-anchored filtering into candidate generation and added backward decomposition and connection checks. In a D6/N25 check, the reported candidate pool fell from 48 to roughly 4-11 candidates per step, exceeding 70% reduction. The table below belongs to this later runner and cannot be spliced into the Week 5 table as a strict ablation. [W6]

Scenario	Expansions U/G	Reduction	Search-space reduction	Guided time (s)
Chain D4 N10	9 / 7	22.2%	21.4%	0.0092
Chain D6 N25	19 / 16	15.8%	13.3%	0.0293
Diamond 3/7 N30	12 / 6	50.0%	50.0%	0.0125
Tree L(3,3) N20	19 / 16	15.8%	13.3%	0.0281

The contemporaneous unguided D6 and tree baselines succeeded in 19 steps, unlike the earlier Week 5 run where both failed at 80. The change in baseline prevents a clean claim that filtering alone “rescued” those tasks, although the later comparison still shows fewer expansions and a much smaller candidate pool. Five backward-primitive unit tests and the 55-test Python unit suite passed at this milestone. [W5, W6]

4 Strategic and bidirectional PLN search

4.1 Tier 2 subgoal guidance

Tier 2 is invoked on stalls and suggests a waypoint for search. The final integration treats the suggestion as a subgoal, never as a ground-truth premise; PLN must derive any bridge from existing evidence. The source milestone contains older architectural prose mentioning premise injection, but its completed integration and measured result explicitly reject axiom injection. [W7, W9, W10]

Case	Plain search	Tier 2 search	Interpretation
Small latent bridge, 10 decoys	passed, 6 expansions	passed, 7; one subgoal	Safe waypoint use; no performance gain
Dead-end-heavy, 8-step budget	failed at 8	deterministic pass at 7	Bounded-success advantage
Same hard case, live provider	failed 3/3	passed 3/3 in 6, 7, 6	3.22-5.86 s assisted; provider dominates

Live proposals included H→N, C→M, and A→C, each established through PLN. The repeat count is three on one constructed hard case, not three independent domains. The result supports strategic recovery under a tight step budget, not lower wall-clock latency. [W7, W9]

4.2 Bidirectional search

The later bidirectional benchmark uses forward PLN application and backward subgoal search, with proof stitching. Its measured benefit is lower costly forward expansion, not an equally large reduction in total search work. [W8, W10]

D	Forward-only steps / nodes / s	Bidirectional total (F+B) / nodes / s	F reduction	Total reduction
6	6 / 6 / 0.0913	5 (2+3) / 7 / 0.0071	66.7%	16.7%
8	8 / 8 / 0.0340	7 (3+4) / 9 / 0.0124	62.5%	12.5%
10	10 / 10 / 0.0605	9 (4+5) / 11 / 0.0190	60.0%	10.0%
12	12 / 12 / 0.0853	11 (5+6) / 13 / 0.0295	58.3%	8.3%

At D=8-12, forward work fell 58-63%, while total expansions fell only 8-13%; generated nodes increased by one in each paired row. The milestone's theoretical O(b^(d/2)) argument is a motivation, not a measured asymptotic result from these four short chains. A later independent audit replayed a bidirectionally stitched proof via PLN.Apply. [W8, W10]

5 External data, scaling, and proof validity

5.1 KGAD similarity pilot

The external kgadexp.metta file is an adjacency-style graph, not ready-made PLN sentences. The loader corrected source syntax in memory, deduplicated 8,280 raw similarityLink occurrences to 3,930 unique edges over 161 nodes, materialized both directions of each Similarity assertion with a shared evidence ID, and assigned the same placeholder STV (0.9, 0.8) to both systems. The graph is complete bipartite: 30 source nodes connect to 131 targets. [KG]

The main comparison used five source nodes, ten targets, 50 unique undirected edges, 100 materialized PLN sentences, five held-out source-to-source queries, a 40-step budget, and 150/150 baseline queues. The baseline was the confidence-driven PLN.Derive loop with no goal-aware hooks. PRISM used a Python-managed frontier but called real PLN.Apply for each accepted transition; Tier 2 and bidirectional search were not used. [KG]

Metric	Confidence-driven PLN	PRISM
Solved	2 / 5	5 / 5
Mean wall time per query	1.04 s	0.51 s
Invalid conclusions	0	0
Synthetic axioms	0	0

Two goals were solved by both systems and had identical strength/confidence, including an observed STV of (0.8143959791953331, 0.6336000000000002). Different legal two-hop routes produced different evidence stamps in this dense graph; equal STVs do not require identical stamps when the path differs. In a separate 100-step, 500/500-queue stress run on three queries, baseline PLN solved 1/3 at 18.38 s mean and PRISM solved 3/3 at 0.51 s mean. [KG]

This is real source data but a small, favorable, single-relation slice with placeholder truth values and short guaranteed proofs. Wall time also compares two complete control implementations, not just two priority functions. [KG]

5.2 Controlled scaling and cross-family transfer

A fixed D5 chain, 30-step budget, five seeded-random baselines, and four fact counts measured pair filtering and bounded search. The reported pair reduction grew from 76.00% at five facts to 99.80% at 55. PRISM succeeded at each listed size; random ordering passed 5/5 at the smallest size but 0/5 at each larger size. [W9]

Facts	Stage 0 pair reduction	Random success	PRISM success	Random / PRISM waste
5	76.00%	5/5	yes	0 / 0
15	97.33%	0/5	yes	29 / 0
30	99.33%	0/5	yes	29 / 0
55	99.80%	0/5	yes	29 / 0

The source reports waste-growth exponents of 0.000 for PRISM and 1.416 for random on this four-point sweep. The random curve is budget-censored at the three larger sizes, so these exponents describe this limited experiment, not general sublinear scaling. With one frozen heuristic and a 20-step budget, PRISM solved chain, diamond, tree, and semantic-gap cases (4/4), while seeded-random ordering solved 0/20 trials over five seeds. [W9]

A separate three-source progression used 100, 1,000, and 5,000 genuine source edges from two WordNet exports and a noisy KG. Only isa was mapped to PLN Inheritance. Each of nine selected slices contained a known real three-edge path with its transitive closure held out; PRISM passed 9/9 in three expansions each. At 5,000 edges, Stage 0 cut a raw 25-million-pair proxy to six candidate pairs. The full graph was not loaded as one unrestricted PLN workload, and these were not random arbitrary KG queries. [W9]

5.3 Independent proof replay and fallback

The Week 10 verifier replayed every recorded derived action using pinned PLN.Apply and compared the term, STV, and evidence stamp with the saved proof. Eight of eight soundness cases passed, totaling 22 of 22 replayed actions. The cases covered chain, diamond, tree, strategic waypoint, bidirectional seam, and 100-edge slices from three external sources. Sixteen of sixteen deterministic failure injections passed: optional scorer, filter, and provider failures took safe fallbacks, and PLN-kernel failure returned no conclusion. Regression snapshots were 121/121 Python tests, 4/4 MeTTa integrations, and 7/7 pinned PLN rule tests. [W10]

This is substantial semantic evidence for the audited paths, not a theorem of soundness for every PLN rule. The broader upstream PLN test.sh was not fully green because examples/Direct.metta had a father/3 error recorded as pre-existing; the milestone gate was the seven pinned rule tests. [W10]

6 Learned Tier 1 v2

The Week 11 model uses trainable atom embeddings and a 128/64 hidden-layer MLP. It trained on 784 labeled candidates from 16 successful PLN-backed searches: 480 positive and 304 negative. Search-run grouping kept 12 runs (569 rows) in training and four runs (215 rows) in validation, without run-ID overlap. Stage 0 was disabled during trace collection to expose multi-candidate frontiers; runtime Stage 0 remains enabled. [W11]

Measurement	Recorded outcome	Meaning
Held-out template classification	215/215 correct; ROC-AUC 1.0	Same small synthetic template families
Held-out shortlist, 43 decisions	v1 100% top-1/top-3; v2 100% top-1/top-3	Parity, not improvement
CPU scorer latency	0.116 ms median; 0.148 ms p95, 1,000 scores	Below 0.5 ms median gate
All atom symbols renamed	58.14% accuracy; 35.25% positive recall	OOD degradation
Predicate and atoms renamed	89.30% accuracy; 19 FN, 4 FP	OOD degradation
External-KG frontiers	69 candidates; 0/9 proof-path positives recalled	Failure on source-derived cases
Adversarial v1 traps	100 frontiers; v1 and v2 both 0% top-1	No correction of v1 errors
30% flipped training labels	95.35% accuracy; ECE 0.318	Calibration deteriorated

The very low validation loss (0.0000468 after 120 epochs) and perfect template accuracy do not establish generalization: v1 was already perfect on those held-out decisions, and v2 receives v1 queue rank as an input feature. V2 did not beat v1 on any reported stress family. It remains an exploratory research result; v1 stays the default. The Week 11 regression snapshot was 126/126 Python, 4/4 MeTTa, and 7/7 PLN rule tests. [W11]

7 OpenPsi planning evaluation

The OpenPsi work ran through the iCog PeTTa-OpenPSI port. Its own getApplicableActions and applyCurrentActions functions produced legal actions and successor states. PRISM selected among them and searched for a goal under a budget; it did not define the action rules or execute the environment. This is a second backend integration, not full feature parity with the PLN stack. [OP]

7.1 Planner reachability and cost

The matched v2 selector experiment used one repository conversation-rule graph, 21 named goals from Conversation-Started, and a 15-transition cap. The selector was the only changed part of the bounded traversal: native findMinDistance solved 3/21 and PRISM solved 20/21. Three timing repeats per goal do not make 63 independent tasks. Reversing insertion order altered some native outcomes, revealing order-sensitive local ties. [OP]

OpenPsi test	Native result	PRISM result	Qualification
Matched v2 selector, 21 goals	3/21 reachable	20/21 reachable	One graph; same transition functions and budget
Fresh-process same-goal check	v1 1/21; v2 3/21	20/21	V1 lacks matched internal search budget
Direct one-step goal	success; 0.023 ms warm	success; 0.043 ms warm	PRISM overhead on easy route
Unreachable goal	dead end; 0.029 ms	dead end; 0.094 ms	No fabricated path
50-decoy two-step case	dead end; 3.67 ms	success; 1.47 ms	Not like-for-like time to solution

In fresh-process per-goal comparisons, median planner-call cold/warm times were 0.423/0.184 ms for v1, 0.496/0.326 ms for v2, and 26.823/3.236 ms for PRISM. Median process wall times were about 132, 135, and 169 ms respectively. PRISM therefore bought reachability at material bridge/startup cost. It failed the Discuss-Chosen-Topic goal at the 15-action limit. [OP]

7.2 Feedback, stochastic simulation, and real action

The original feedback loop incorrectly marked an invalid native path as goalMet and left demand unchanged. After local OpenPsi-side path-validation and bounded-demand fixes, both a constructed fixture and a repository conversation fixture showed native v1 returning an invalid route and competence staying 0.10, while PRISM returned a valid route and competence rose 0.10→0.55. This is a corrected symbolic feedback result, conditional on a valid plan. It is not direct evidence that user-facing actions completed. [OP]

Action-success assumption	v1 completion	v2 completion	PRISM completion
1.0	4.76%	14.29%	95.24%
0.9	3.81%	11.64%	49.07%
0.7	2.36%	7.38%	16.72%

The stochastic figures above come from 1,000 simulated episodes per goal per mode using assumed independent action-success probabilities; they are not calibrated OpenPsi outcomes. In the one effectful repository ping-pong test, native v2 and PRISM both chose act-ping: both returned the ball at motivation 1.0 and both missed at 0.1. The actual-action comparison is a tie. The adapter also did not support the repository trader schema, whose candidate generator returned no actions for its 24 rules. [OP]

8 Conclusions and evidence boundary

- Supported for the tested PLN tasks: goal-aware candidate selection improves bounded search on noisy chains; indexed filtering can remove many irrelevant premise pairs; explicit search policies reduce selected expansion counts; and accepted later proofs remain grounded in PLN.Apply in the audited cases.
- Supported for the tested OpenPsi planning task: PRISM improves path-valid bounded reachability on one conversation-rule graph and can affect a corrected symbolic demand loop when its plan is valid.
- Not established: a universal speedup, arbitrary-query or million-edge AtomSpace performance, superiority of neural v2, a general ECAN comparison, or better real OpenPsi agent outcomes.
- Priority follow-up: independent graph/query families, randomized ordering and larger repeated samples, matched ECAN baselines where available, and a neural training set with unfamiliar vocabularies and cases where v1 actually errs.

Source index

W1-W11: milestones/milestone_week1.md through milestone_week11.md. KG: milestones/kgadexp_pln_vs_prism_comparison.md. OP: milestones/openpsi_evaluation.md. C: milestones/prism_results_consolidated.md (cross-check and numerical ledger). Later machine-readable reference files are under prism/benchmarks/results/reference/, including week9_scaling_generalization.json, week9_external_kg_scaling.json, week10_soundness_robustness.json, week11_tier1_v2.json, and the OpenPsi evaluation JSON files. These are historical recorded experiments, not one newly rerun benchmark campaign.